

OpenGL Graphics & Transformations Projects

3D Flag Scene: A 3D environment rendering a stylized Union Jack flag hoisted on a pole. It incorporates structured 3D primitives, camera setups via `gluLookAt`, and custom bounds-checked physical transformations.

The project utilizes a standard **3D Geometric Transformation Pipeline** via the OpenGL Matrix Stack (`glPushMatrix` and `glPopMatrix`) to position, scale, and manipulate objects relative to each other:

- **Translation (`glTranslatef`):** Used to position the ground, build the multi-tiered base, offset the pole, position the cone finial, and dynamically move the entire asset across the screen (tx) or slide the flag up and down the pole (flagYOffset).
- **Scaling (`glScalef`):** Used to distort primitive shapes (e.g., stretching a `glutSolidCube` into a tall, thin flagpole or flat base steps) and to dynamically scale the overall flag size based on user input (+ / - keys).
- **Rotation (`glRotatef`):** Used precisely once to rotate the `glutSolidCone` by -90° around the X-axis so that it points vertically upward instead of along the default Z-axis.

Implementation Strategy

The scene relies on hierarchical transformations to keep elements anchored together:

- **Global Position:** A master `glPushMatrix()` applies the global translation (tx, 0.0f, tz) so that pressing 'l' or 'r' moves the bases, pole, finial, and flag seamlessly as a single compound asset.
- **Flag Pivot Alignment:** Inside `drawUKFlag()`, the flag geometry is centered around the origin (0,0,0). To attach it naturally to the side of the pole, a compound transformation sequence is used:
 1. Translate to the top of the pole plus the vertical offset: `glTranslatef(poleThickness / 2.0f, 4.3f + flagYOffset, 0.0f);`
 2. Apply dynamic scale factors: `glScalef(flagSizeX * 0.8f, flagSizeY * 0.8f, 1.0f);`
 3. Shift the flag's left edge to the pivot point: `glTranslatef(1.4f, -0.8f, 0.0f);`
- **Depth Layering:** To render the complex, overlapping geometry of the Union Jack without clipping artifacts (Z-fighting), incremental, tiny offsets in the Z-axis (e.g., 0.01f to 0.05f) are explicitly assigned to each overlapping polygon layer.

Ethiopian Airlines Logo: A 2D vector-rendering engine that maps Bezier curves and uses the GLU Tessellator to render complex, filled geometries with interaction support (rotation, translation, and scaling). 2D Transformations Used

The application applies three fundamental affine transformations to manipulate the rendered geometry relative to its canvas:

- **Translation:** Moves the logo's position in 2D space. It handles interactive movement (locX, locY) and structural centering offsets (translateX, translateY).
- **Rotation:** Rotates the logo around a localized center point using rotationAngle.
- **Scaling:** Adjusts the uniform size of the contours via scaleFactor.

Implementation Details

The transformations are implemented using fixed-function OpenGL matrix operations inside the display() callback. To prevent the logo from spinning wildly out of bounds or scaling away from the screen center, **pivot-point transformation** logic is used to isolate the changes around the logo's core coordinates (roughly centralized around (12.5, 12.5)):

C++

```
glTranslatef(locX, locY, 0.0f);  
  
glTranslatef(12.5f + translateX, 12.5f + translateY, 0.0f); // Pivot shift  
  
glRotatef(rotationAngle, 0.0f, 0.0f, 1.0f);           // Apply Rotation  
  
glScalef(scaleFactor, scaleFactor, 1.0f);             // Apply Scaling  
  
glTranslatef(-12.5f, -12.5f, 0.0f);                   // Restore Origin
```